

CLASS IMBALANCE AND FEW-SHOT LEARNING WITH OBJECT DETECTION ALGORITHM YOLO3

Student Name : Vu Duc Nghia
Student ID : 2611002V

Dissertation

Abstract

Data plays a crucial role in machine learning as well as computer vision, both quality and quantity of data has significant impacts to performance of models. One popular problem with data quality is that one or more classes have dominant representation while other classes have much lower representation. Besides this, in some situations data is not easy to collect and we have only few samples to work with, these scenarios are called few-shot learning. So how should we work in case data is imbalanced or has only a limited number of samples. This dissertation introduces image augmentation methods to address both issues. We use a subset of Camera trap dataset, that have both features “highly imbalanced” and “very few samples” for some classes. We also use object detection algorithm YOLO3 as a model to work with our data. By applying image augmentation techniques, we increase the performance of the model from just 0.064mAP to 0.283mAP when working with imbalance class, and from 0.114mAP to 0.377mAP when dealing with few-shot learning. Beside addressing problems with data, we also dive deep into implementation of the YOLO3 algorithm, a member of the one-stage object detection family YOLO. We mention details from model architecture, image generator, training process to evaluation method.

Contents

1	Background.....	4
1.1	Computer vision overview	4
1.2	Object Detection.....	4
1.3	YOLO algorithms.....	6
2	Convolutional Neural Network and its components.....	7
2.1	What is a convolution?	7
2.2	Convolutions in Tensorflow:.....	8
2.3	Activation, max pooling and upscaling.....	10
3	YOLO Algorithms	12
3.1	Concepts in Object Detection:.....	12
3.1.1	Bounding box:.....	12
3.1.2	Intersection over Union (IoU):.....	13
3.1.3	Anchor box:	13
3.1.4	Non-max suppression:	14
3.2	YOLO Details:	14
3.2.1	Prediction output:.....	14
3.2.2	Loss Function:.....	16
3.3	Evaluation metrics:.....	17
4	Implementation.....	19
4.1	Generator.....	19
4.2	YOLO network.....	20
4.3	Training module:	22
4.4	Evaluating module:	23
5	Data, Class imbalance and few-shot learning.....	25
5.1	Camera trap datasets.....	25
5.2	Project dataset	25
5.3	Class Imbalance:	27
5.4	Few-shot learning:.....	28
5.5	Data Augmentation:	29
6	Results, Analysis and Discussion	31

6.1 Imbalanced test:..... 31

6.2 Few-shot learning..... 33

6.3 Conclusion:..... 36

6.4 Limitation: 36

6.5 Future work: 36

1 Background

1.1 Computer vision overview

Computer vision is an area of artificial intelligence which enables computers and systems to derive insightful information from digital images, videos and take actions or recommendations based on that information. The ultimate goal is to make computers able to see, observe and understand visual inputs as humans.

Computer vision performs similarly to human eyes and it works by training machines with a lot of data or images. It runs analysis of data to find features, patterns from images. Two essential technologies are utilized to achieve this is deep learning and convolutional neural network (CNN). A CNN model will conduct convolution operations to output predictions about what it is sighted.

The first experiment of computer vision took place in 1959 and then four years later, in 1963, computers could transform two-dimensional images into three-dimensional forms. The invention of new technologies ‘optical character recognition’ (OCR) and ‘intelligent character recognition’ (ICR) in 1974 led to a new milestone that computers could recognize text in any font as well as hand-written text. In 1982, neuroscientist David Marr introduced new algorithms for machines to detect edges, corners and curves and similar basic shapes (IBM, n.d).

By 2000, the focus of study was on object detection. That is a field of artificial intelligence where computers are able to recognize and locate positions of different objects like car, person, sofa, animals in images. In 2010, ImageNet dataset which contains millions of labeled images became available and provided a foundation for CNN and deep learning model used today (IBM, n.d).

1.2 Object Detection

Object detection has been witnessing a rapid revolutionary change in the field of computer vision since 2020. That combines object classification tasks and object localization tasks make it one of the most challenging topics in the domain of computer vision. Object detection is much more challenging than classification and localization (Verdhan, 2021, p.142-144).

Classification only identifies whether a single object exists in an image or not. Classification and localization is a bit more challenging, besides classification tasks, it has to identify the location of the object as well. However, both classification, and classification and localization only apply for a single object in an image. While object detection has to detect the object and its location for multiple objects (Verdhan, 2021, p.142-144). The image below (figure 1) distinguishes between these techniques.

The focus of object detection emerged from 2000 and rapidly grew to achieve significant breakthroughs thanks to the advances of deep learning and computing power of GPU’s. Object detection has been used in many real-world applications, such as healthcare monitoring, autonomous driving, video surveillance, anomaly detection, and robot vision (IBM, n.d).

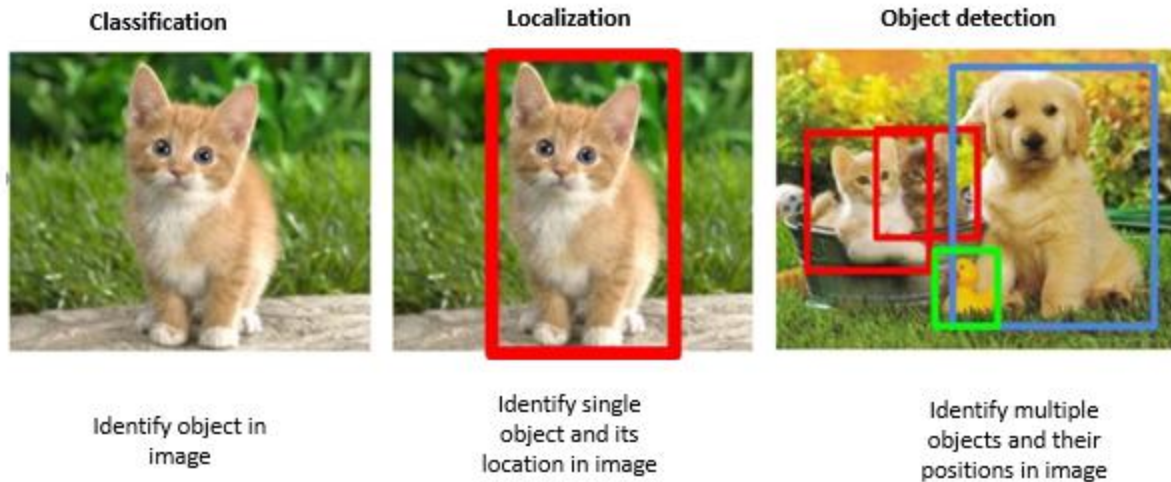


Figure 1-Distinguish Classification, Classification and Localization, Object Detection

Object detection can be performed using either traditional image processing techniques or modern deep learning networks. Image processing techniques do not require historical data and it is unsupervised in nature. These techniques are restricted to multiple factors, such as complex scenarios. And deep learning methods, which depend on supervised training. This method is more robust than image processing techniques but it requires more computing power. However, this problem is mitigated nowadays due to the release of stronger processing units GPU (IBM, n.d).

Deep learning detection methods started to flourish in 2014 with the two most categories of detection algorithms: One-stage algorithms and two-stage algorithms (Jiang, et al, 2019, p.21-30).

The most important one-stage algorithms are:

- YOLO (2016),
- SSD (2016)
- RetinaNet (2017)
- YOLOv3 (2018)
- YOLOv4 (2020)
- YOLOv5 (2020)
- YOLOR (2021)

And the most important algorithms for two-stage algorithms are:

- RCNN (2014)
- Fast RCNN
- Faster RCNN (2015)

G-RCNN (2021)

The two-stage detectors will have two steps; the first step finds regions of interest (use object regional proposal) then use these cropped regions for classification (second step). This makes two-stage detectors run longer and slower but it achieves high accuracy. While one-stage detectors omit the object regional proposal step, it directly predicts object bounding boxes for images in a one-stage fashion (Jiang, et al, 2019, p.21-30). This leads to a simpler and faster model architecture, although it can sometimes struggle to be flexible enough to adapt to arbitrary tasks.

1.3 YOLO algorithms

YOLO or You Only Look Once for short is our concentration in this dissertation, which is a real-time object detection that identifies specific objects in videos or images (Redmon, et al., 2016). The first version of YOLO was created in 2016, then it has continued to evolve with different versions, such as YOLOv3 (by Joseph Redmon) in 2018, YOLOv4 (by Alexey Bochkovskiy) in April 2020, YOLOv5 (by Glenn Jocher) in June 2020. The main focus of this dissertation is on YOLOv3, this is not the latest algorithm but It will provide a good foundation to study other deep learning algorithms. We will examine thoroughly the underlying techniques and implementation of this model. We also assess how performance of the model is impacted by imbalance classes or few-shot learning issues.

2 Convolutional Neural Network and its components

Convolutional Neural Network is the core component of the YOLO3 model. This chapter we will dive deeper into elements of this network.

2.1 What is a convolution?

Convolution is an operation that combine two functions:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)G(t - \tau)d\tau$$

Convolutions have many useful properties when processing images. The above formula is for continuous space. In the scope of this project, we will only use discrete convolutions which is defined as:

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f(m)g(n - m)$$

Where g is known as the kernel of the convolution.

Above definitions are for one dimensional convolution. Next, we will explore two dimensional convolutions which are used for matrix or 2D arrays. Suppose, we have matrix (Image) M and Kernel-K, the convolution of Kernel-K over image M is shown below:

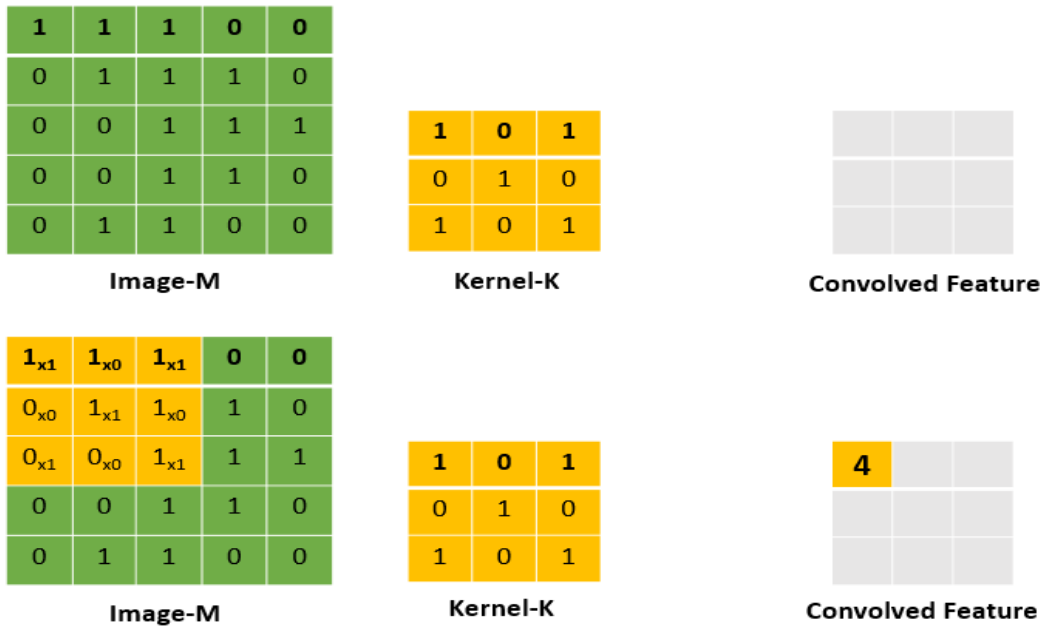


Figure 2 - Convolution on 2D Image

To perform the first convolution, the Kernel-K is put on top-left part of Image-M then do element wise multiplication then sum up to get result of 4:

$$1 \times 1 + 1 \times 0 + 1 \times 1 + 0 \times 0 + 1 \times 1 + 1 \times 0 + 1 \times 1 + 0 \times 0 + 1 \times 1 = 4$$

Similarity, we can calculate the second, third and so on by sliding the Kernel over the Image from the left to the right and from the top to the bottom

1	1 _{x1}	1 _{x0}	0 _{x1}	0
0	1 _{x0}	1 _{x1}	1 _{x0}	0
0	0 _{x1}	1 _{x0}	1 _{x1}	1
0	0	1	1	0
0	1	1	0	0

Image-M

1	0	1
0	1	0
1	0	1

Kernel-K

4	3	

Convolved Feature

1	1	1	0	0
0	1	1	1	0
0	0	1 _{x1}	1 _{x0}	1 _{x1}
0	0	1 _{x0}	1 _{x1}	0 _{x0}
0	1	1 _{x1}	0 _{x0}	0 _{x1}

Image-M

1	0	1
0	1	0
1	0	1

Kernel-K

4	3	4
2	4	3
2	3	4

Convolved Feature

Convolutional Neural Network to advantages of convolutions when working with structured data. Lower levels of the CNN employ convolutions to detect fine scale features of the data. These features are then passed through the network to reveal more complex patterns.

2.2 Convolutions in Tensorflow:

Tensorflow has command to perform convolutional layers in 1, 2 or 3 dimensions. These commands are:

`tf.keras.layers.Conv1D`

`tf.keras.layers.Conv2D`

`tf.keras.layers.Conv3D`

Some key parameter regarding to these convolutional layers

Number of filters:

This is the number of kernels used in convolutional layers, the larger this number the more parameters will be to tune and then the network also be able to detect a larger variety of patterns and features.

Kernel size:

That is the size of the kernel, it can be identified by a tuple of integers, for example (3,3) or (3,5). It is standard to use odd numbers for kernel size so that the center point is clear. Most CNN architectures tend to use a kernel size of 3.

Padding and strides

These two parameters control how the kernel tile on the data array and what happen in the edges. When we do a convolution on the area which close to the edge of image (array), if we want the centre of kernel over the top of cell in the corner of image then the kernel will past the edge of image. If we do not want the kernel go out of the edges, then the output will be smaller size than original array and cells in the edges are used less than cells placed firmly inside the images, that could lead to losing some information. Therefore, we have two options to address this issue and these are termed “same” and “valid”.

If we choose “same” then output will have the same size with input, in this case we need to pad amount of zeros to outside of original array in order to maintain the size of input at output.

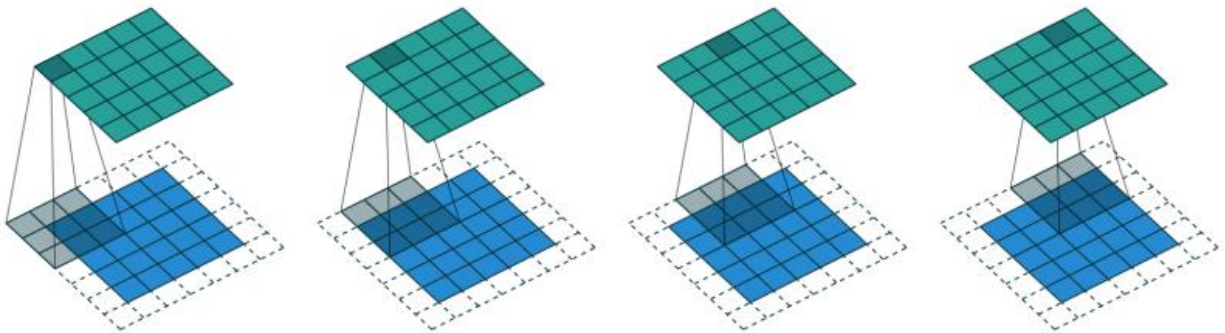


Figure 3 - Same padding

If we choose “valid”, then the kernel just passes over the cells in the original array and no need to pad zeros to the edges of the image. As a result, the output size will be smaller than the input size.

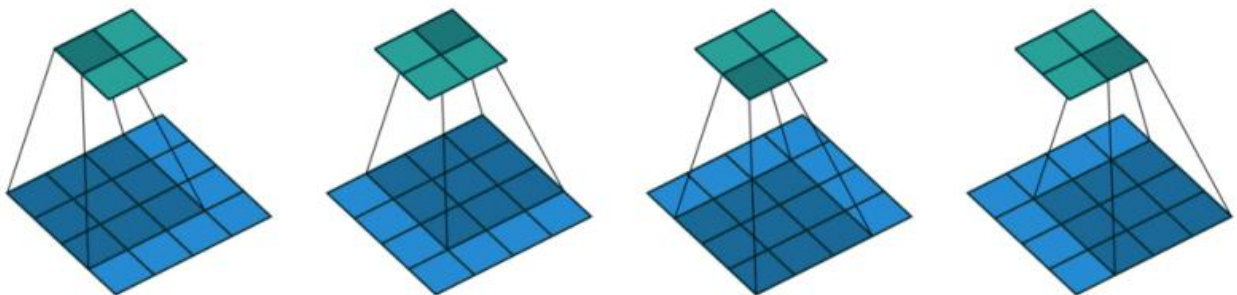


Figure 4-Valid padding

Stride defines how many rows or columns that kernel will move from one step to the next step, usually we use default value at one then kernels will pass through all entries of the original array. Increasing stride will decrease the size of output, however it is common to keep it at default value of 1 and reduction size of array will be handled by max pooling layer.

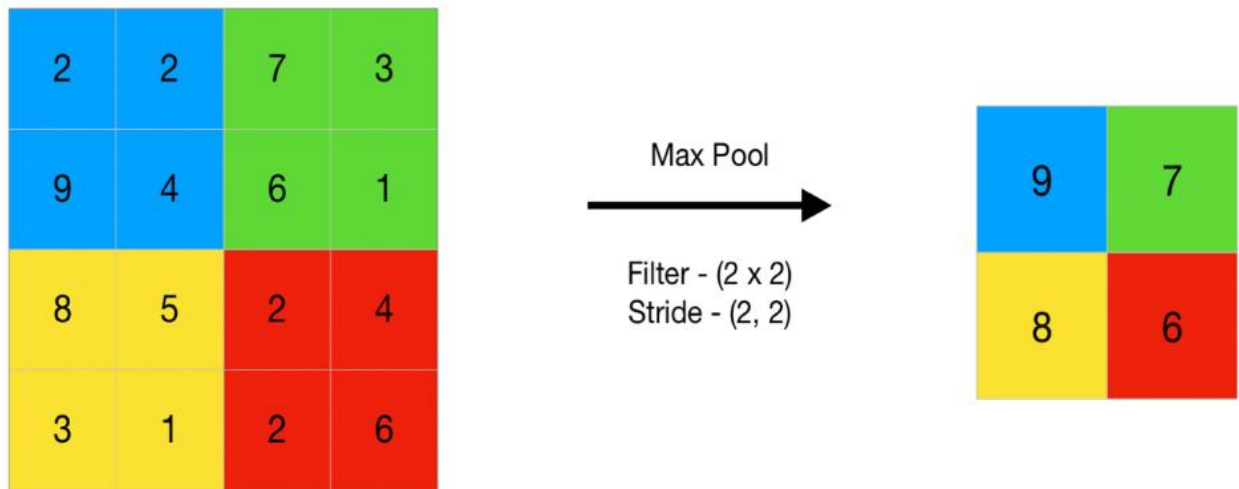
2.3 Activation, max pooling and upscaling

Activation

Activation functions are used to transform entries of output. Some popular activation functions are sigmoid, ReLu, leaky-ReLu, softmax. In this dissertation, we mainly use the leaky-ReLu function.

Max pooling

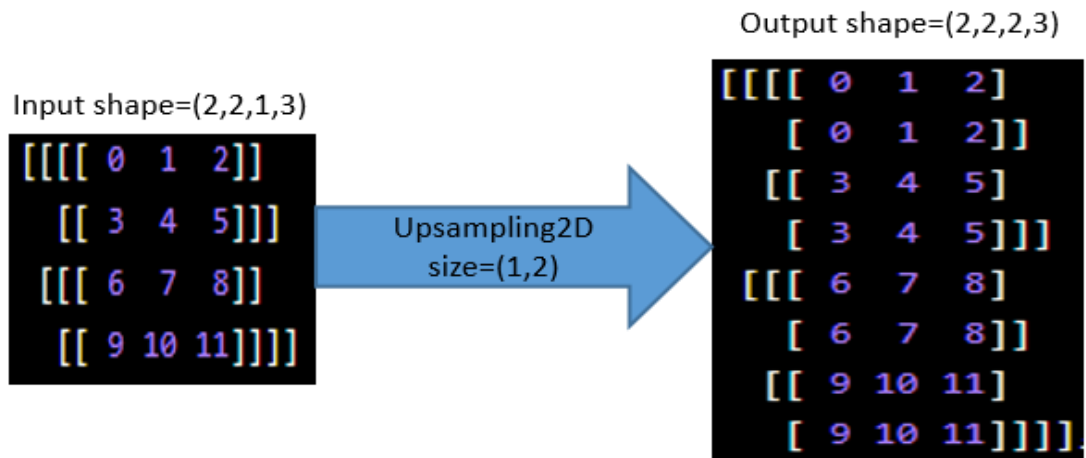
Max pooling is a pooling operation that returns a maximum value over an input window (size defined by pool_size) for each channel of the input. The window is shifted by strides along each dimension. Max-pooling helps to reduce the size of input then reduce computational workload but it still maintains patterns of images (O'Shea and Nash, 2015).



Upsampling

If max-pooling will down sample or reduce the size of input, then upsampling is a technique to increase the size of input by repeating corresponding rows and columns. Keras provides three layers to handle this task: UpSampling1D, UpSampling2D and UpSampling3D. It applied for different input sizes. In our dissertation, we will only use the UpSampling2D layer.

For UpSampling2D, the input size must have 4 dimensions, shape in format of (batch_size, height, width, channels) or (batch_size, channels, height, width). Default shape is (batch_size, height, width, channels), which means that data format follows the 'channels_last' option. The parameter **size** is a tuple of two integers (f1, f2) which will control the up sampling process. The elements in dimension **height** will be repeated f1 times and elements in dimension **width** will be repeated f2 times. So the output of Upsampling2D will have shape (batch_size, height*f1, width*f2, channels).



3 YOLO Algorithms

"You Only Look Once" (YOLO) is a popular algorithm in object detection which has high prediction accuracy and especially fast prediction then it is suitable for real time application (Redmon, et al., 2016). This part will provide important concepts as well as mathematical details used in the algorithm.

3.1 Concepts in Object Detection:

3.1.1 Bounding box:

In the object detection field, a bounding box is used to identify location of objects as well as object type. The location is represented by a rectangle which is identified by top left coordinates (x_{min} , y_{min}) and bottom right coordinate (x_{max} , y_{max}). The rectangle sometimes is characterized by centered coordinates together with its width and height [x_c , y_c , width, height]. In many scenarios, we need to convert from one format to another to fit to your model, the transformation is simple and based on following equations:

$$x_{min} + x_{max} = 2 * x_c$$

$$y_{min} + y_{max} = 2 * y_c$$

$$x_{max} - x_{min} = \text{width}$$

$$y_{max} - y_{min} = \text{height}$$

The following image illustrates a bounding box and its coordinate. In this image, [x_{min} , y_{min} , x_{max} , y_{max}] = [170, 120, 616, 320] and [x_c , y_c , width, height] = [393, 220, 446, 200].

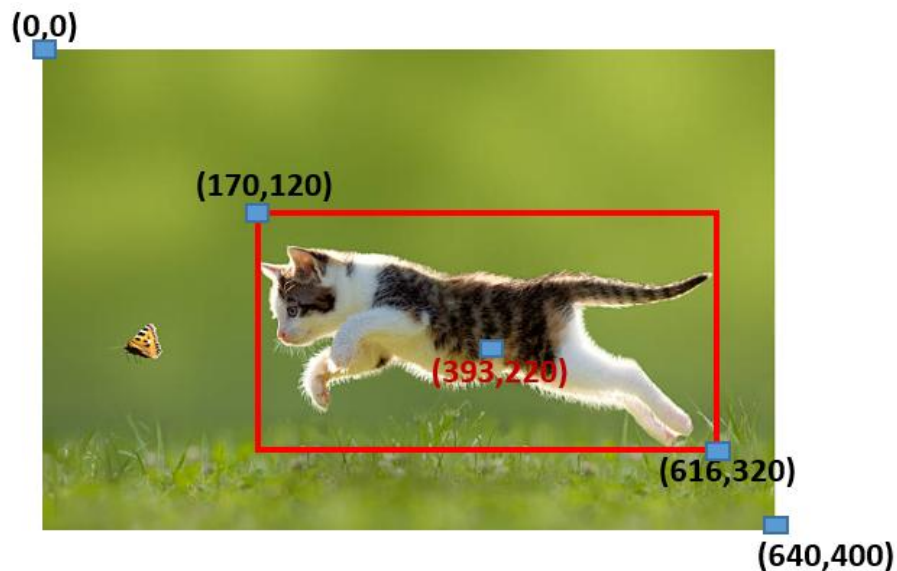


Figure 5-Bounding Box example

3.1.2 Intersection over Union (IoU):

The IoU is an evaluation metric to measure the accuracy in object detection. Given a ground-truth bounding box and a predicted bounding box. IoU can be computed as ratio of overlap area to union area (Verdhan, 2021, p152-153):

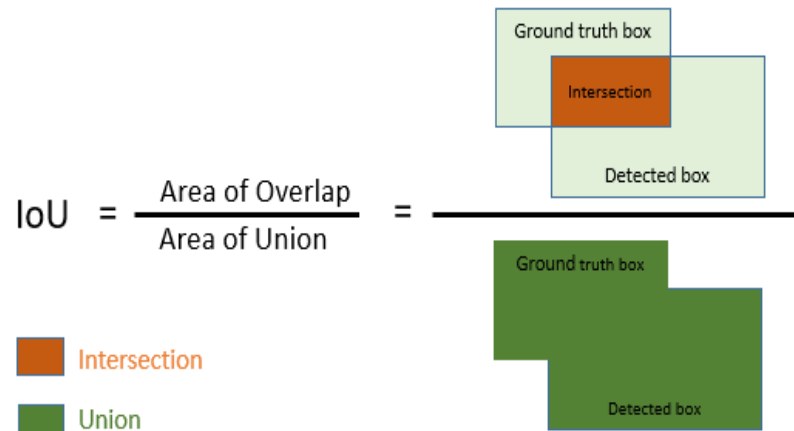


Figure 6-Intersection over Union

3.1.3 Anchor box:

In object detection, the cell that contains the center of the bounding box will be responsible to detect the object. There are scenarios where we have different objects with centers in the same cell then the system is unable to detect them all. Then we use one more concept that is called Anchor box to ease these situations. YOLO3 uses three anchor boxes for each cell, each Anchor box represents for one object, and it is represented by a pair of numbers, for example (4,5), (20,30), (100,200) and so on. The anchor box helps to capture the scaling and aspect ratio of the objects that we want to detect (Verdhan, 2021, p155-156).

In the image below, the midpoints of person and car are in the same cell, the bounding box of person looks similar to anchor box 2 and the bounding box of car is similar to anchor box 1. Therefore, we assign the car to anchor box 1 and person to anchor box 2. In the YOLO3 model, we use three anchor boxes for each YOLO layer, so as we have three layers then nine anchor boxes are needed.

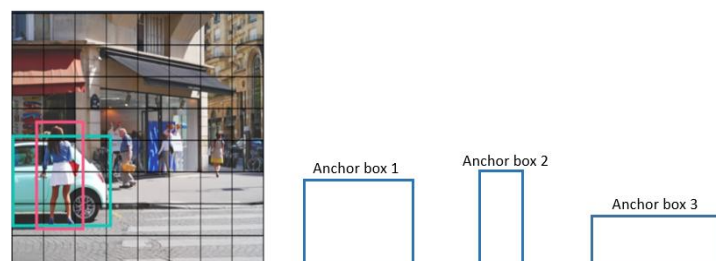


Figure 7-Example of anchor boxes

3.1.4 Non-max suppression:

The algorithms may have multiple detections of the same objects then non-max suppression is a method to ensure that each object only be detected once. Let see an example below, there are up to three bounding boxes to detect a cat with different probabilities at 0.7 0.8, 0.9, non-max suppression will help to discard two extra boxes and keep the remaining box with highest probability.

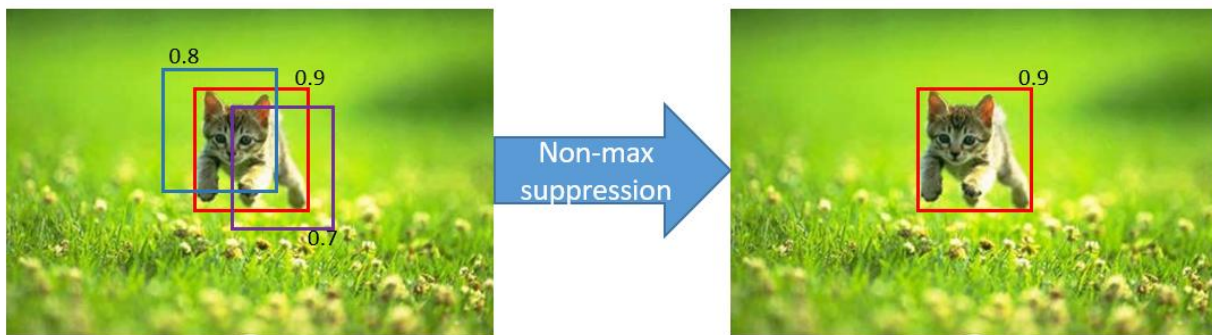


Figure 8-Nonmax suppression

Here is how the algorithms work:

First, algorithms will discard all boxes with probability p_c less than a threshold which is usually at 0.5, then for remaining boxes we perform following tasks (Verdhan, 2021, p154-155):

- Pick the box with highest probability as the output
- Discard any remaining box with IoU is greater than 0.5 with the box in previous step
- Repeating above two steps but leaving out all boxes that we already pick as the outputs.

3.2 YOLO Details:

3.2.1 Prediction output:

YOLO model works by dividing the image into $N \times N$ grid cells, each cell having the size of $S \times S$. It means that the image before feeding to the network must be resized into $(N \times S, N \times S)$ shape. In YOLO3, S has three values 8, 16 and 32. A cell containing the midpoint of an object will be responsible to detect that object. Each cell can detect up to three objects corresponding to three different anchor boxes. Each object is represented by a vector which contains information about the predicted object, the length of this vector is $(5 + nb_class)$, where nb_class is the number of classes in the dataset (Redmon and Fahadi, 2018): $(t_x, t_y, t_w, t_h, p_c, C_1, C_2, \dots, C_n)$

Where:

t_x, t_y : the center coordinate of objects

t_w, t_h : the width and height of bounding box

p_c : the confidence that there is an object in a cell for some anchor box. (p_c is between 0 and 1)

$C_1, C_2, \dots, C_n$: the likelihood that the object belongs to class 1, class 2, ... or class n

$$C_1 + C_2 + \dots + C_n = 1$$

Therefore, if we divide the image into $N \times N$ grid then the output will have size of $N \times N \times 3 \times (5 + \text{nb_classes})$. Below image demonstrated for the case N equal to 4.

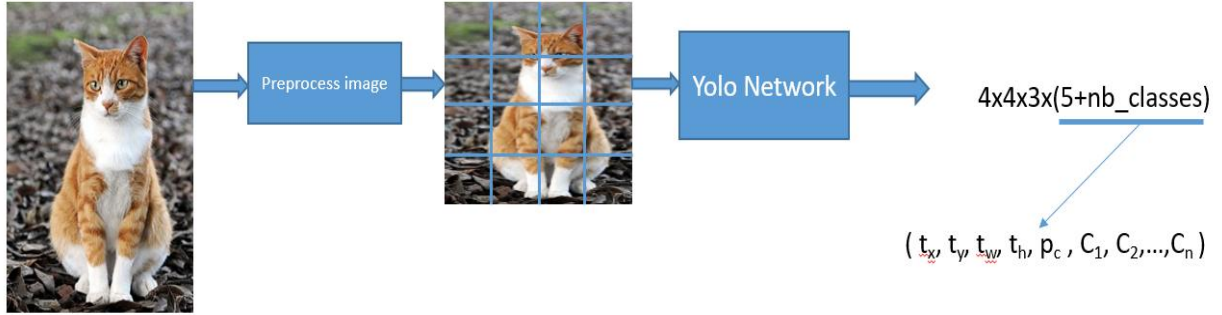


Figure 9-Simple data flow in YOLO3

It is noted that t_x, t_y, t_w, t_h are not raw values, it is already processed. Let say the left corner of a cell have coordinates (c_x, c_y) on grid size scale and the bounding box prior has width and height p_w, p_h (or assigned anchor box) then we have following formula (Redmon and Fahadi, 2018):

$$b_x = \sigma(t_x) + c_x$$

$$b_y = \sigma(t_y) + c_y$$

$$b_w = p_w e^{t_w}$$

$$b_h = p_h e^{t_h}$$

σ : sigmoid function

b_x, b_y : centered coordinates of bounding box on (0, N) scale

b_w, b_h : width and height of bounding box on standardized image scale.

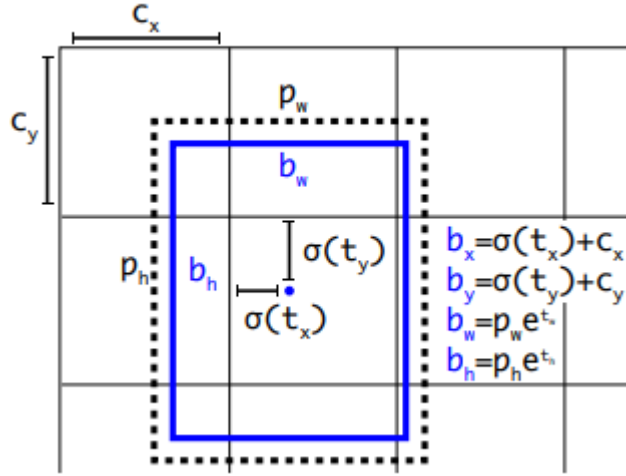


Figure 10-Bounding boxes with dimension priors and location prediction (Redmon and Fahadi, 2018)

3.2.2 Loss Function:

Classification Loss: This penalize for incorrect prediction of class label.

$$L_{\text{class}} = -\lambda_{\text{class}} \sum_{i=0}^{N^2} \sum_{j=0}^B I_{ij} \log (p_{\text{groundtruth_class}})$$

Localization Loss (Regression Loss): This penalizes for misidentifying location of objects

$$L_{\text{regression}} = \lambda_{\text{regression}} \{ \sum_{i=0}^{N^2} \sum_{j=0}^B I_{ij} [(b_{x,ij} - \widehat{b_{x,ij}})^2 + (b_{y,ij} - \widehat{b_{y,ij}})^2] + \sum_i \sum_{j=0}^B I_{ij} [(t_{w,ij} - \widehat{t_{w,ij}})^2 + (t_{h,ij} - \widehat{t_{h,ij}})^2] \} * (2 - \text{box_area}_{ij})^2$$

Confidence Loss: This penalizes for two cases:

- + No objects but model predicts there are objects in image
- + Having objects but predicted confidence score is low.

$$L_{\text{confidence}} = \sum_{i=1}^{N^2} \sum_{j=0}^B [I_{ij} (p_{c,ij} - \widehat{p_{c,ij}}) * \lambda_{\text{obj}} + (1 - I_{ij}) * D_{ij} * \lambda_{\text{no_obj}}]^2$$

Total Loss Function:

$$L = L_{\text{class}} + L_{\text{regression}} + L_{\text{confidence}}$$

Where:

N, B: grid size and number of anchor boxes per cell, B is equal to 3 for YOLO3

I_{ij} : equal to 1 if an object exists in cell i and anchor j, otherwise 0

$\bar{I}_{ij}$: is inverse to I_{ij} , it equal to 0 if no object exists in cell i and anchor j

$p_{\text{groundtruth_class}}$: probability of ground truth class predicted by model, it lies between 0 and 1

$b_{x_{ij}}, b_{y_{ij}}$: center coordinates of bounding boxes at cell i for anchor box j on $(0, N)$ scale ($\text{sigma}(t_{xy}) + c_{xy}$)

$\widehat{b_{x_{ij}}}, \widehat{b_{y_{ij}}}$: predicted center coordinate of bounding boxes at cell i for anchor box j on $(0, N)$ scale

$t_{w_{ij}}, t_{h_{ij}}$: the width and height of bounding boxes in cell i , anchor box j (t_w, t_h)

$\widehat{t_{w_{ij}}}, \widehat{t_{h_{ij}}}$: the predicted width and height of bounding boxes in cell i , anchor box j (t_w, t_h)

box_area_{ij} : the ratio between area of bounding box to area of image

D_{ij} : equal to predicted confidence score $\widehat{p_{c_{ij}}}$ if IOU of predicted bounding box to ground truth is less than a threshold (iou_threshold) for all ground truths of image, otherwise it is equal to 0.

$\lambda_{\text{class}}, \lambda_{\text{regression}}, \lambda_{\text{obj}}, \lambda_{\text{no_obj}}$: some preset constants to regulate loss components.

3.3 Evaluation metrics:

Object detection uses average precision(AP) and mean average precision(mAP) as a metric to evaluate its performance. While average precision is used for estimating prediction performance on a single class(label), mAP is used for evaluating performance on the whole model which includes all classes. AP and mAP is developed based on two concepts which are recall and precision.

For each class/label, recall and precision are defined as below (Jiao, et al., 2019):

$$P = \frac{TP}{TP + FP} = \frac{TP}{\text{all detections}}$$

$$R = \frac{TP}{TP + FN} = \frac{TP}{\text{all ground-truths}}$$

TP: true positive, model predict correct object

FP: false positive, model predict an object but actually such object does not exist

FN: false negative, there is an object but model is not able to detect it

These values depend on the confidence score threshold, high threshold leading to low recall as number of TP reduces while number of FN increases. In contrast, low threshold leads to high recall because more TP and less FN. The precision-recall (PR) curve represents the relationship between precision and recall over different values of threshold.

$\text{AP}@ \alpha$ is the Area Under the precision-recall curve(AUC-PR) at threshold α of IoU (which determines whether two boxes overlap). For example, AP50 is AP calculated at IoU threshold of 0.5, and AP75 is calculated at IoU threshold of 0.75. The formula to calculate AP from vector recall R and vector precision P as below:

$$AP@{\alpha} = \sum_i (R_{i+1} - R_i) * \max (P_{i+1}, P_{i+2}, \dots, P_n)$$

Mean average precision (mAP) are simply the average of AP over all classes:

$$mAP@{\alpha} = \frac{1}{n} \sum_{i=1}^n AP_i \quad \text{for } n \text{ classes.}$$

4 Implementation

The project is organized into several modules:

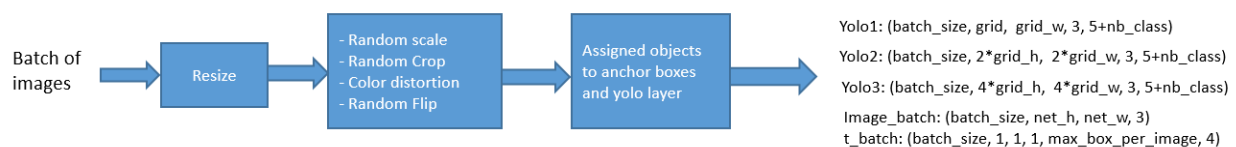
- Generator: convert data into standard format, perform internal image augmentation and generate batches
- YOLO: this is the YOLO model constructed in Keras, received batches from Generator and output the prediction
- Training: manage training process
- Evaluation: manage evaluation or validation process
- Utils functions: these functions support for above modules

We will only focus more details on key modules that are essential for YOLO3. As YOLO3 is a complex model so understanding deeply these modules are great foundations to explore other models in the future. I try to explain and present ideas of the code as well as how to implement it so readers can easily capture.

4.1 Generator

This module reads raw images and their annotations then generates a batch of images in a standard format. Images will go through a series of random augmentation steps: scale, crop, flip, color distortion. Final images will have some pre-specific sizes. Objects in images are also classified or assigned to a specific anchor box and YOLO layer (small, medium or large) in the generator module. In order to determine which anchor box for a specific object, YOLO will calculate IoU between the object and all anchor boxes then the anchor box with the highest IoU with the object will be assigned for that object. Information about which grid cell containing the bounding boxes is also identified, the coordinates of the center box and its height, width is also converted to standard format. Coordinates of center boxes are converted to grid size scale (for example, image is divided into grid 10x10 then the scale will be between 0 to 10). And the height and width of the bounding box are converted to natural logarithm of its height, width over assigned height and width of bounding box (or call prior bounding box).

The main idea of this module can be explained by follow work flow.



The output from generator module will have:

YOLO1: objects are assigned to YOLO1 layer (large objects)

YOLO2: objects are assigned to YOLO2 layer (medium objects)

YOLO3: objects are assigned to YOLO3 layer (small objects)

Image_batch: all images in the batch after augmentation and resized

t_batch: contain information about all boxes, this will serve for later calculation

Detail code is in generator.py file.

4.2 YOLO network

This module handles YOLO network and loss function. The YOLO model is constructed from two main components: Convolutional block and layer YOLOLayer. We will examine more details about these two modules.

Convolutional block:

Convolutional block is a small convolutional network which includes a series of groups of layers, each group of layers having one convolutional layer and may have other layers such as Zeropadding2D, BatchNormalization and ReLU connected in that order. The convolutional block has a structure as in the below diagram, a shortcut connection is added at the last two groups. The shortcut connection will prevent vanishing gradients during training a deep network (Ebrahimi and Abadi, 2018). The depth of the Convolutional block will depend on the number of convolutional layers in it. We will use the notation Conv_B(n) to indicate this block has n convolutional layers.



YOLOLayer:

YOLOLayer is a custom layer which is built on top of “Layer class” in Keras. This custom layer plays as a loss function, which does various transformations on input data and calculates loss components. Images after going through a series of convolutional blocks will output three tensors in the form of (batch_size, grid_h, grid_w, 3*(5+nb_classes)) which are then fed to YOLO layers. It is noted that grid size for each YOLO layer will be different, grid size of YOLO layer two double grid size of YOLO layer one, and layer three double YOLO layer two. They are then reshaped into (batch_size, grid_h, grid_w, 3, 5+nb_class). The last dimension indicates the center coordinates of the bounding box, confidence of object and probability of each class (t_x , t_y , t_w , t_h , p_c , C_1 , C_2 , ..., C_n). The figure below depicts the interaction between generator module, convolutional network and YOLO layers:

YOLO3 model has three components that already explained detail in chapter 2, we will revisit and highlight key points when computing these components:

Regression components:

Which panelize for difference between predicted locations and true locations:

$$L_{\text{regression}} = \lambda_{\text{regression}} \left\{ \sum_{i=0}^{N^2} \sum_{j=0}^B I_{ij} [(b_{x_{ij}} - \widehat{b_{x_{ij}}})^2 + (b_{y_{ij}} - \widehat{b_{y_{ij}}})^2] + \right. \\ \left. \sum_{i=0}^{N^2} \sum_{j=0}^B I_{ij} [(t_{w_{ij}} - \widehat{t_{w_{ij}}})^2 + (t_{h_{ij}} - \widehat{t_{h_{ij}}})^2] \right\} * (2 - \text{box_area}_{ij})^2$$

Box_area is used to control the impact of object sizes, when objects are small then box_area are small and vice versa. I_{ij} are determined at the beginning of the training process (by generator module). The indicator I_{ij} helps to limit the calculation to only cells (and anchor box) which actually have objects.

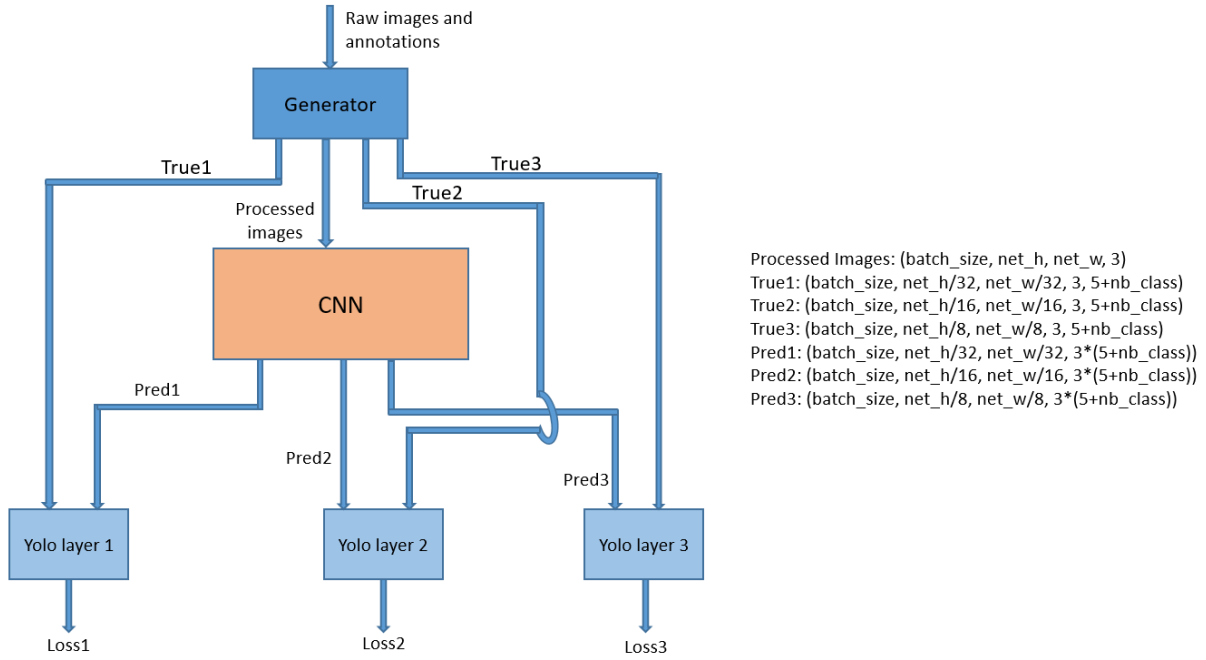


Figure 11-Block diagram of YOLO3

Label component:

This component penalizes the difference between predicted classes and actual classes

$$L_{\text{class}} = -\lambda_{\text{class}} \sum_{i=0}^{N^2} \sum_{j=0}^B I_{ij} \log(p_{\text{groundtruth_class}})$$

The calculation limits only cells (and anchor) which actually have objects. So how $p_{\text{predicted_class}}$ is calculated in this formula? We recall that the predicted probability that output from convolutional network in the form $(C_1, C_2, \dots, C_k)$ where k is number of class then:

$$p_{\text{groundtruth_class}} = \frac{\exp(C_*)}{\sum_{i=0}^{k-1} \exp(C_i)}$$

Where * is the index of ground truth label, it is between 0 and k-1.

We can see that the higher $p_{\text{groundtruth_class}}$ the lower the loss component and that is also our target as higher $p_{\text{groundtruth_class}}$ the more confidence for our prediction.

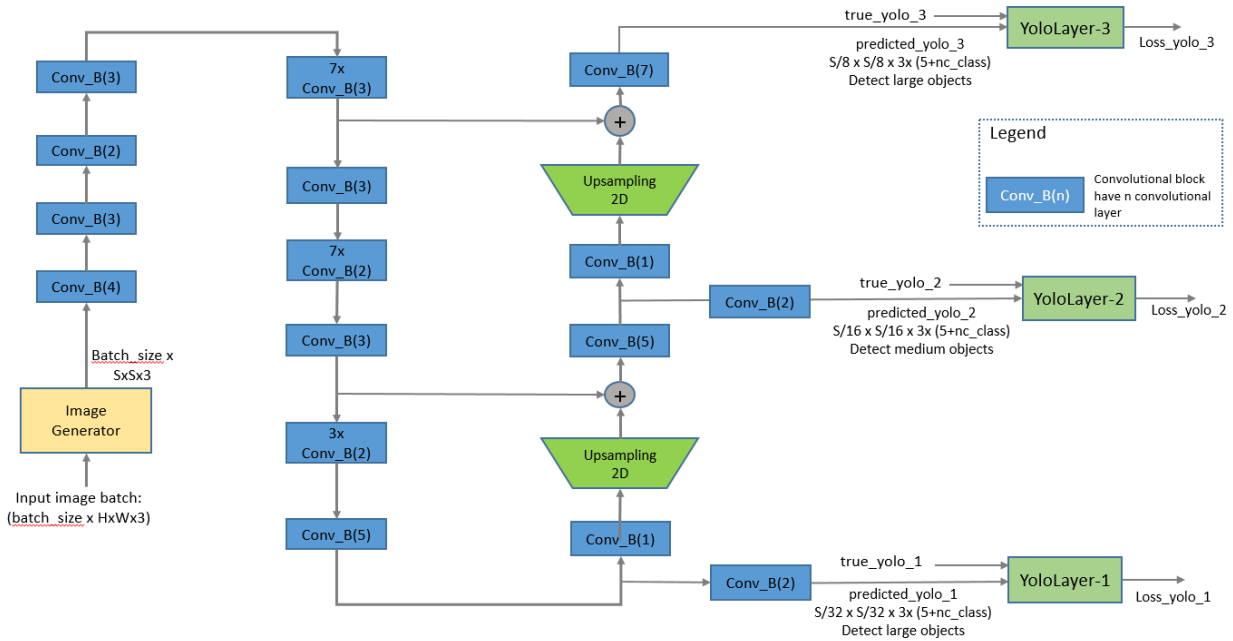
Confidence loss component:

Confidence loss has two parts:

$$L_{\text{confidence}} = \sum_{i=1}^N \sum_{j=0}^B [I_{ij} (p_{c_{ij}} - \widehat{p}_{c_{ij}}) * \lambda_{\text{obj}} + (1 - I_{ij}) * D_{ij} * \lambda_{\text{no_obj}}]^2$$

Where D_{ij} is equal to $\widehat{p}_{c_{ij}}$ if IoU between box in cell i, anchor j and all ground truths are less than 0.5, otherwise D_{ij} is equal to zero.

The underlying idea in this formula is that, cells that actually contain objects we want predicted confidence scores to be as high as possible to one and cells that do not have objects then we want predicted confidence scores in these cells to be as low as possible to zero. That will drive $L_{\text{confidence}}$ to lower values.



4.3 Training module:

This module will manage the training process. It interacts with the generator, YOLO network and configuration parameters input by users. Some important features designed for this module to control training process are:

- Automatically update learning_rate during training: learning_rate will automatically reduce to factor*learning_rate if loss do not reduce a predefined value after certain number of epochs

- Early stopping: the training process will stop if loss do not reduce a specific amount after a number of epochs

There are various parameters we need to set-up for the training process. Basically, we have parameters for the model, for training, for validation and internal augmentation (using for the generator). These parameters are wrapped into a json configuration file. The figure below shows an example of such parameters.

```
[
  "model" : {
    "min_input_size":      384,
    "max_input_size":      384,
    "anchors":              [2,2, 9,10, 16,15, 16,31, 24,79, 36,39, 52,84, 100,125, 195,314],
    "labels":               ["Elephant", "Grants Gazelle" , "Giraffe"]
  },

  "train": {
    "train_file":          "train/fewshot_crop.csv",
    "train_times":          1,
    "batch_size":           8,
    "learning_rate":        1e-4,
    "nb_epochs":            50,
    "warmup_epochs":        2,
    "ignore_thresh":        0.5,
    "gpus":                  "1",
    "grid_scales":           [1,1,1],
    "obj_scale":             5,
    "noobj_scale":          1,
    "xywh_scale":           1,
    "class_scale":          1,
    "tensorboard_dir":       "logs",
    "saved_weights_name":    "Fewshot_crop.h5",
    "debug":                 true
  },

  "valid": {
    "valid_file":           "valid/fewshot_valid.csv",
    "valid_times":          1
  },

  "augmentation":{
    "Flip":true,
    "Scale": [0.6,1.4],
    "Color_distort": [18,1.5,1.5]
  }
]
```

4.4 Evaluating module:

Once the training process is completed and we have trained weights, the evaluating module will evaluate performance of the model by conducting model performance on validation or test set.

An image before going through the model will pass through **preprocessing** block for standardization, after pass through the model, we will have three tensors which presents for three YOLO layers:

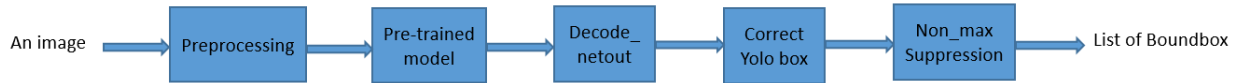
(1, grid_h, grid_w, 3, 5+nb_class)

(1, 2*grid_h, 2*grid_w, 3, 5+nb_class)

$(1, 4*grid_h, 4*grid_w, 3, 5+nb_class)$

where the last dimension has the structure like this $(t_x, t_y, t_w, t_h, p_c, C_1, C_2, \dots, C_n)$. These outputs will pass through a **decode** block to decode information and then to get a list of boundingbox. Each boundingbox has the format of $(x_{min}, y_{min}, x_{max}, y_{max}, objectness, class)$ where objectness represents how much confidence an object is detected, class represents label of object. This output then passes another block call `correct_yolo_box` to update the changes made by **preprocessing** block. A `non_max` suppression will apply the list to clear all overlap boxes.

Above process can be shown as diagram below:



Detail operations of decode block as below:

Apply sigmoid on t_x, t_y, p_c

Apply softmax operation on $C_1, C_2, \dots, C_n$

Calculate objectness, that is product the outputs of two previous steps

Filter out all bounding boxes with objectness less than 0.5

Getting coordinates of bounding box $x_{min}, y_{min}, x_{max}, y_{max}$ on relative scale to image height and width, so range of these values will vary from 0 to 1.

Create boundingbox object in the form of $(x_{min}, y_{min}, x_{max}, y_{max}, objectness, class)$

Put all boundingboxes into a list as output of the decode block.

Once we get a clean list of boundingboxes, we will compare this list with ground truths, we will have information about True Positive(TP), False Positive(FP), False Negative(FN) which are necessary for computing Average Precision(AP) and then Mean Average Precision(mAP) for the model. Detailed explanations for nonmax-suppression and how to calculate mAP is presented in chapter 3.

5 Data, Class imbalance and few-shot learning

5.1 Camera trap datasets

Camera trap dataset are collected automatically by cameras at different locations in order to monitor and study animal populations as well as their behavior in nature by biologists. This is a source of images for computer researchers to evaluate how well their models can generalize to unseen locations. The key challenge to the generalization issue is that many unseen places taken at day and night time are not present in the training set (Beery, n.d).

Images in the datasets are at high resolution, it can contain animals of interest or other objects without interests (people, vehicles). The animals of interest can be so small that it is even hard to find them in the images. Some provided annotations are incorrect but the quantity is small (Beery, n.d).

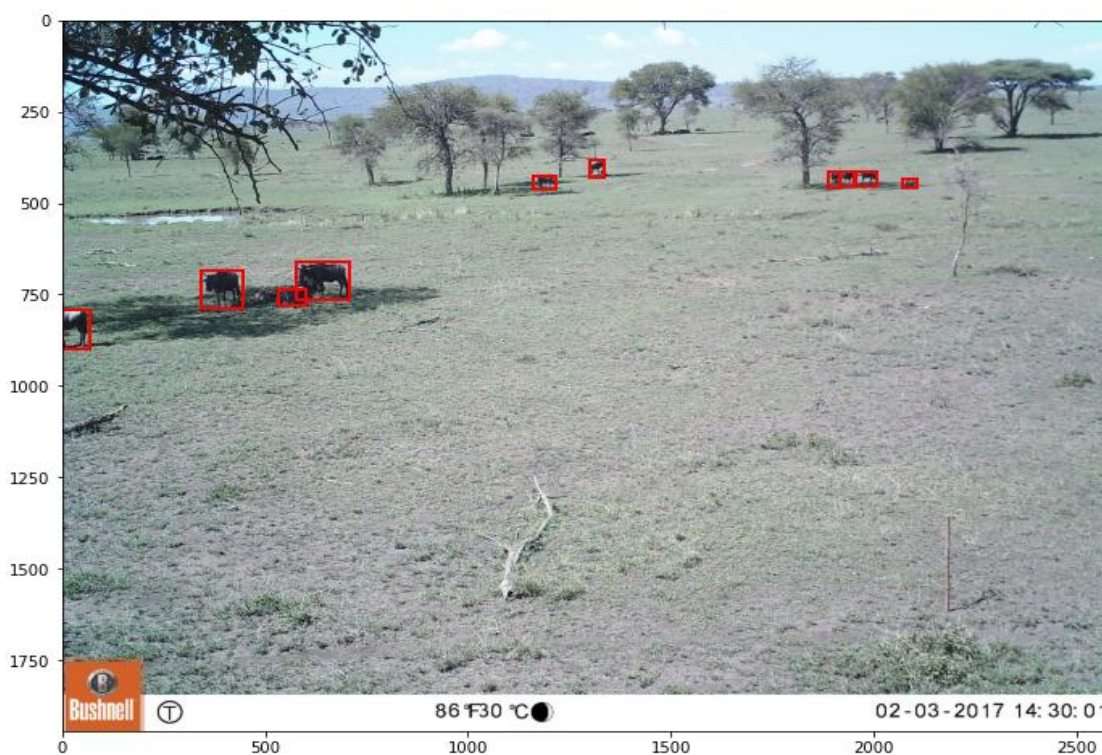


Figure 12-Sample of camera trap image

5.2 Project dataset

There are different versions of the camera trap dataset. In this dissertation, we use a version provided by University of Glasgow Lab. We do not use the whole dataset for research but use a subset of it to investigate the class imbalance and few-shot learning in the object detection field. Dataset has four different classes: Elephant, Giraffe, Wildebeest and Grants Gazelle in 4186 images (before splitting into train and validation). The image sizes are (2592,1944) and (3264, 2448). The below plot shows number of annotations and number of images for each class. (It is

noted that each image can contain several classes). It can be seen that the class ‘Wildebeest’ is dominant in both factors and the dataset is highly imbalanced.

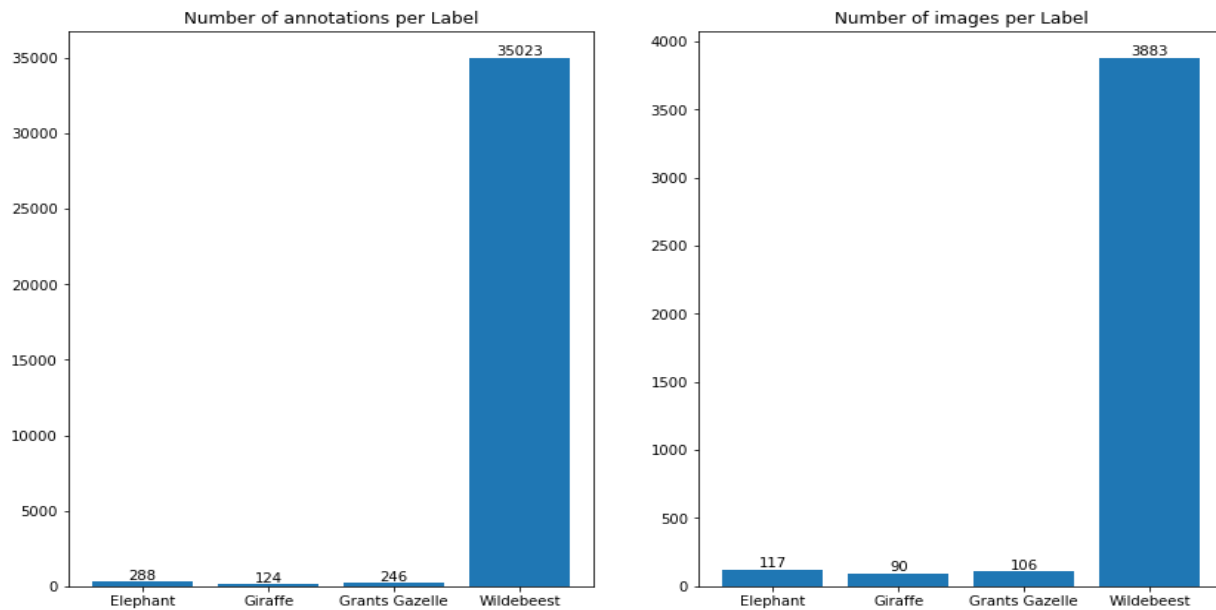


Figure 13 - Data distribution among class in Cameratrapp dataset(training+valid)

In order to ensure the generalization of the object detection model, we will split this dataset into a train-validation set following ratio of 1:9. Then the validation set will have much more images that ensure the validation process has better accuracy. Number of images and annotations per class are indicated in below plot, class “Wildebeest” still dominate other classes.

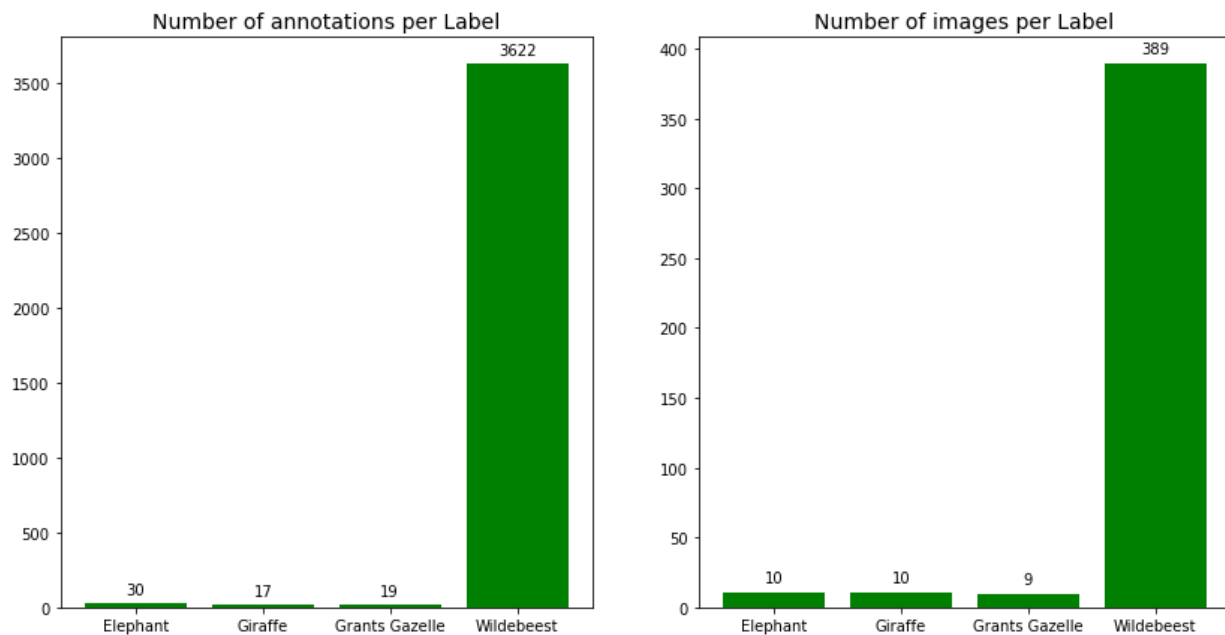


Figure 14 - Data Distribution among class in training dataset(imbalance)

And the below chart shows distribution for validation dataset, it has the same distribution as training dataset but the quantity is around nine times larger so the results on validation set give us more confidence.

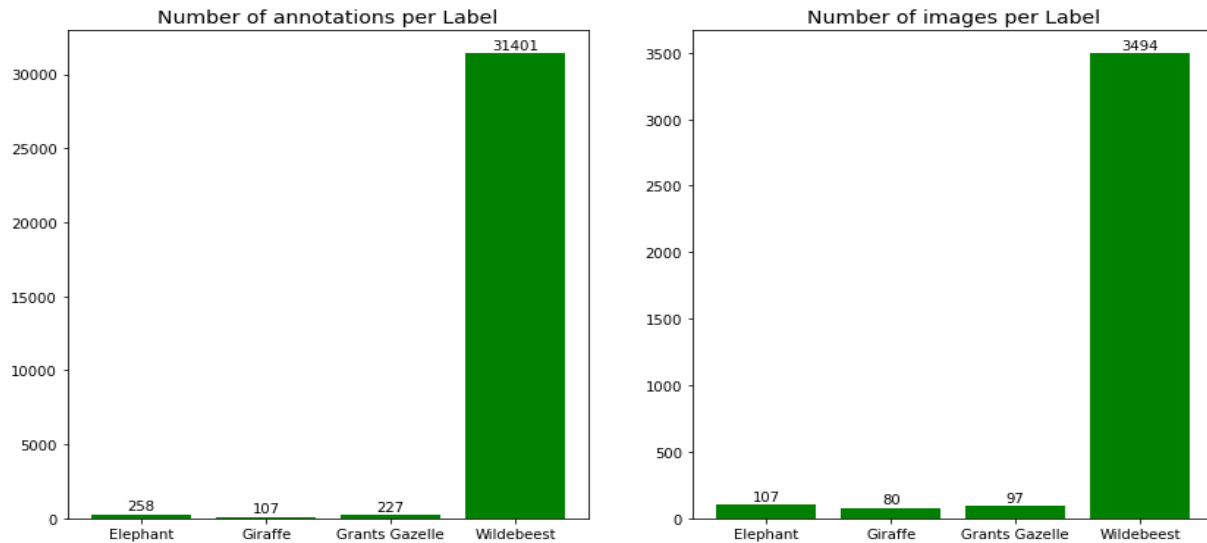


Figure 15-Data Distribution among class of validation dataset(imbalance)

5.3 Class Imbalance:

Class imbalance can have negative impacts on performance of models (Park, et al., 2021). It happens when some classes have dominant representation in the dataset and some classes have a much lower number of samples in the dataset (Ochal, et al., 2021). For example, in our cameratrap training set, “Wildebeest” are over represented while others are under-presented in our samples. There are several methods to address this issue (Ochal, et al., 2021):

- ✚ Collect more images, data for under-represented classes to have a more balanced dataset. This is always the best choice but in many situations, data is not available or easy to collect. So we will not consider this solution in this project.
- ✚ Just duplicate (up-sampling method) all under-represented classes to get balance between classes. This is a simple method but it actually does not add new information for the model and may cause our model heavily over fit to training data and less generalization which lead to poor prediction performance. We will not consider this in the project as well.
- ✚ Remove part of over-represented classes (down-sampling method)
- ✚ Create synthetic image: This method will create new images based on existing data. The new images will be modified from original images but still retain features, patterns of original ones. There are two popular approaches for this method:
 - Image augmentation: this approach utilizes traditional but high efficient techniques to generate more synthetic data for under-represented classes (Yang, et al., 2022):
 - ✓ Flipping image

- ✓ Cropping image
- ✓ Scaling image
- ✓ Color distortion
- ✓ Translating image
- ✓ Rotating image
- Generative adversarial network (GAN)

In this dissertation, we will only use image augmentation to generate variants of original images.

5.4 Few-shot learning:

As a general rule to achieve high performance for a supervised learning task is to feed the model with as many as possible samples. However, in many situations we just have a limited number of samples as collecting more examples are difficult, high cost or even they are not available. Few-shot learning research generally concentrates on image classification as visual information is easy to acquire. However, other areas in computer vision also get attention from the community such as object detection, image segmentation (Parnami and Lee, 2022). It is worth mention here that in case we have only one sample for each class then the learning process called **one-shot learning**

A common question is how to deal with situations where we have limited data? The figure 15 provides an overall picture to the few-shot learning area. We see that there are two general approaches are Non-Meta-Learning and Meta-Learning. Most of recent progress takes place in Meta-learning (Parnami and Lee, 2022). We will not discuss detail about each approach because it exceeds the scope of the project which is emphasized on YOLO3 and object detection.

For our project, we stick to the Generative few-shot learning approach where we need to create more samples from existing data. Common methods for this approach are image augmentation and generative adversarial network(GAN). Our concentration is on image augmentation. It is worth noting that for classification problems we often need only two to five images per class. For object

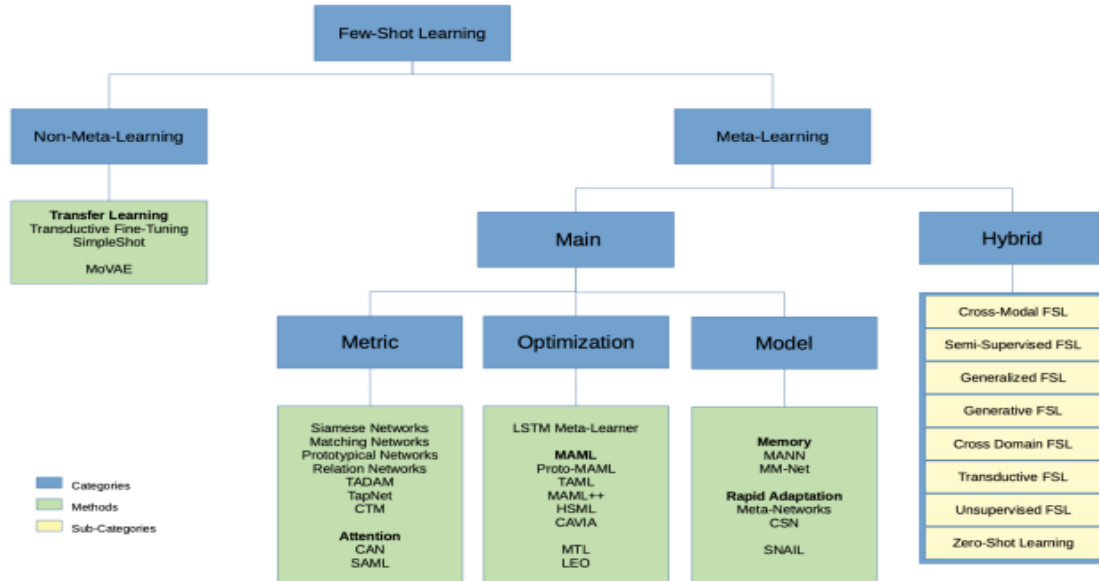


Figure 16-Approaches to few-shot learning

problems, the task is more complicated as it combines classification and regression problems then the number of images will be higher to achieve meaningful improvement.

5.5 Data Augmentation:

Data augmentation is a set of techniques to generate more images from original images, it is commonly used in computer vision to create training samples (Yang, et al., 2022). It is very useful in situations where we have a limited number of samples or it is difficult or costly to collect new samples. YOLO3 is equipped with the image augmentation module which helps to diversify our training sets then improve performance of the model. We introduce here some common techniques but very useful in practice:

Flip: new image got by flip the original image

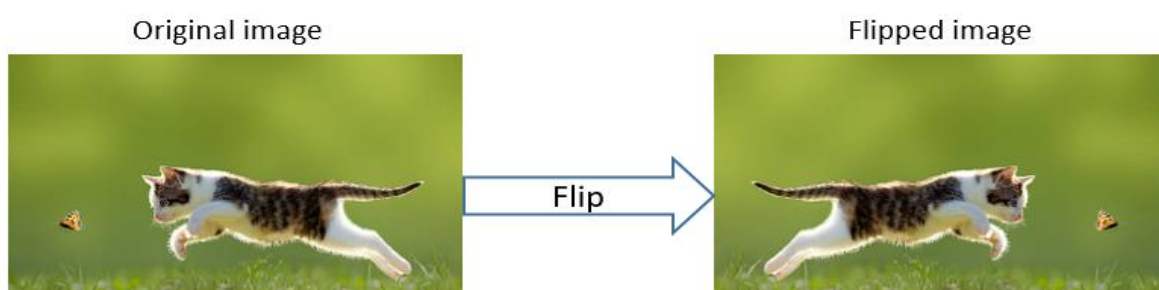


Figure 18 - Flip an image

Figure 17-Approaches to Few-Shot Learning

Crop: New image got from a part of original image

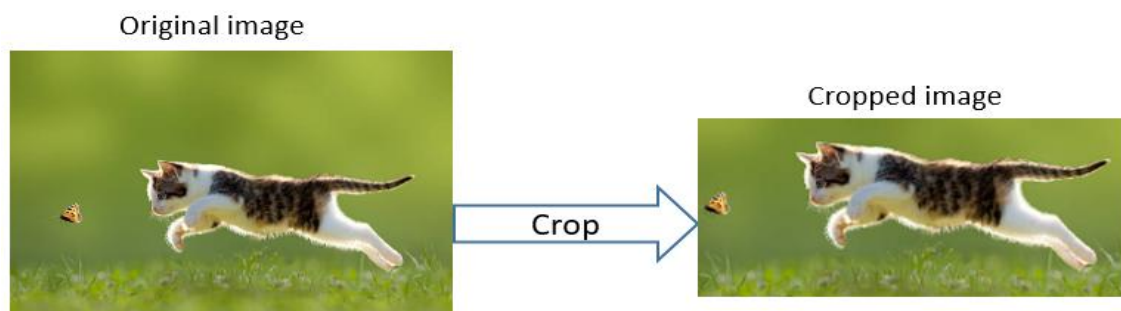


Figure 19 - Cropping an image

Color distortion: New images get from changing color of original image which help to diversity weather and lighting conditions where picture is captured.

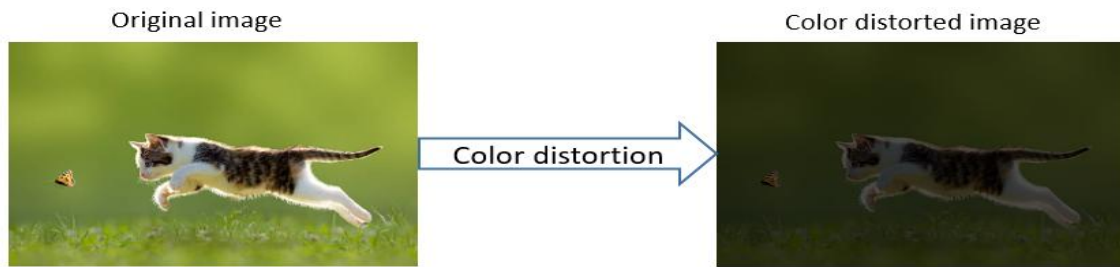


Figure 20 - Color distortion

6 Results, Analysis and Discussion

At this stage, we have already built the model and prepared data. We can start to train the network and analyze its performance on different aspects.

6.1 Imbalanced test:

We first train our model on the imbalanced dataset with four classes that are already prepared in chapter 3 and then validate the model on validation set (please find data distribution for training and validation set in part 5). Below is validating result:

Table 1 – Imbalanced – Validating results when using original training data (not augmented)

Class	AP
Elephant	0.0011
Giraffe	0.0811
Grants Gazelle	0.0000
Wildebeest	0.1741
Mean Average Precision(mAP)	0.0641

The result surprises us as the mean average precision of the model is very low, just at 0.0641mAP and performance on classes are highly imbalanced. AP on Wildebeest is significant while for other classes are negligible. We can doubt that there is something wrong with data or implementation of the model. But it is obvious that there is still some signal on Wildebeest class. So potential problems may be in training data. Recall that the training dataset is highly imbalanced. Most of the images and annotations are for “Wildebeest” class that is why performance on this class is better than the other.

In order to address this issue, we will use image augmentation techniques to generate more samples for under-represented classes Elephant, Giraffe, Grants Gazelle in order to achieve a more balanced dataset. More specifically, we will crop 48 different parts of every image that contains labels for Elephant, Giraffe, Grants Gazelle. It is noted that some images may contain several classes rather than only one or they even contain “Wildebeest” class as well.

Here are some explanations about how to create 48 new samples from an existing image. We denote each group of four numbers (x_1, y_1, x_2, y_2) which vary between zero and one represents coordinates (in relative to width and height of an existing image) of top-left corner and bottom-right corner of new image position in old image. Suppose w, h are width and height of existing image (in pixel) then cropped image is shown in figure-21. Then each i value belongs to $\{0,1,2,3,4,5,6,7\}$. We create six group numbers which then create 48 new samples from an existing image.

$(i/20, 0, 1, 1)$

$(0, 0, 1-1.2*(8-i)/20, 1)$

$(i/17, i/25, 1, 1)$

$(0, 0, 1-(8-i)/20, 1-(8-i)/25)$
 $(i/30, i/35, 1-(8-i)/30, 1-(8-i)/35)$
 $(i/20, i/20, i/20+0.5, i/20+0.5)$

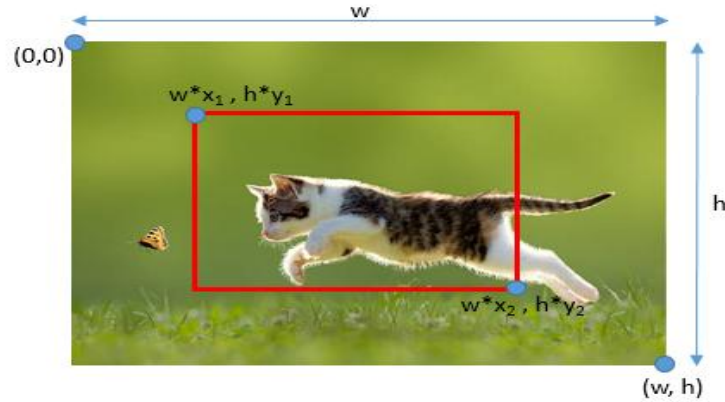


Figure 21-A new sample

As shown in figure 22, the number of images between classes are now more balanced than the original set. Number of annotations per class is still imbalanced but the ratio between low representation classes over the high representation class is much better.

Training on augmented dataset gives much better performance. The improvements occur in all classes. The strongest enhancement takes place in class Grants Gazelle where AP increases from zero to 0.5054 then followed by class Elephant where the number rises from just 0.0011 to 0.3510. Wildebeest class also witnessed the increase of AP, although we do not intentionally augment for the images containing such class. It is obvious that image augmentation proves to be very effective for this scenario.

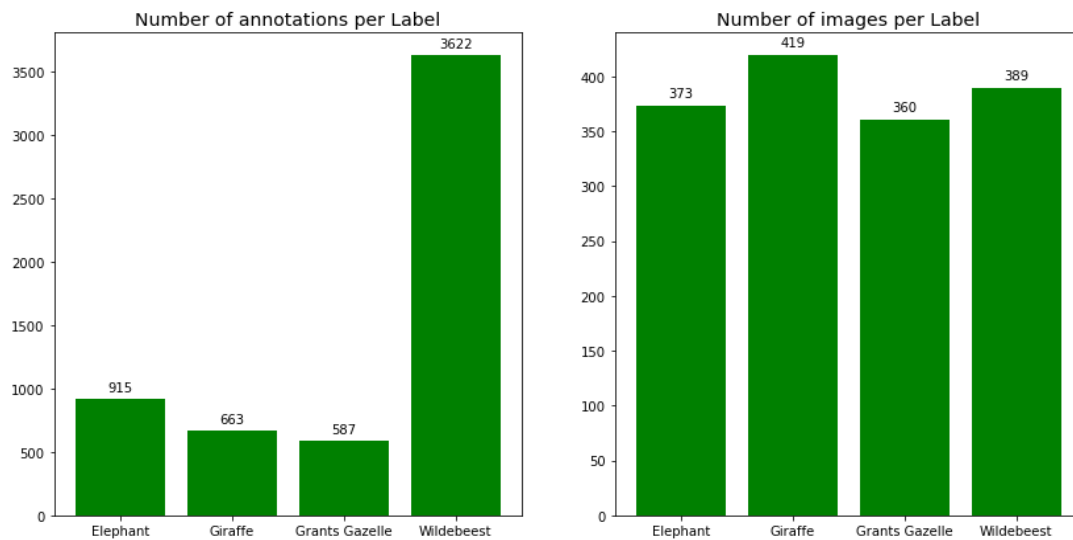


Figure 22 – Imbalance-Data Distribution among class in the augmented dataset

Table 2 – Imbalanced - Validating results when training data do and do not augmented

Class	AP	
	Original	After augmentation
Elephant	0.0011	0.3510
Giraffe	0.0811	0.2444
Grants Gazelle	0.0000	0.5054
Wildebeest	0.1741	0.4304
Mean Average Precision(mAP)	0.0641	0.3828

6.2 Few-shot learning

Next, we will examine the few-shot learning issue. Data for this part will inherit from the imbalanced part by removing the “Wildebeest” class. So the remaining part will have only three classes and number of images or annotations per class are quite limited for object detection problems but imbalance do not appear in this dataset anymore.

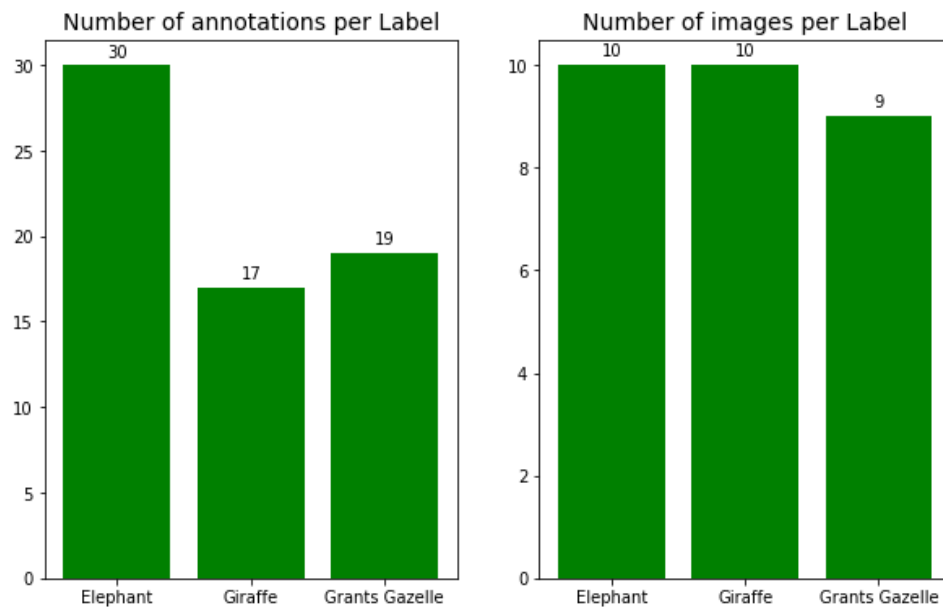


Figure 23 – Few-shot - Data Distribution of training dataset before augmentation

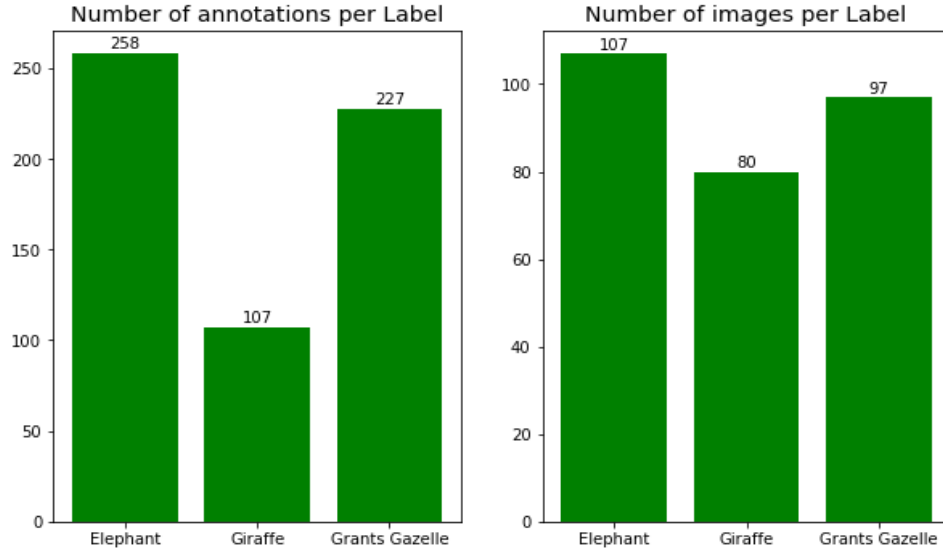


Figure 24-Few-shot - Data Distribution of the validation dataset

Training the model on this data then validating on the validation set gives quite poor performance as shown in table-3. Strongest signal happens on class Grants Gazelle while results on class Elephant and Giraffe is very poor which lead to overall performance of model is at only 0.1143.

Table 3 - Few-shot – Testing results when training data is not augmented

Class	AP
Elephant	0.0281
Giraffe	0.0266
Grants Gazelle	0.2882
Mean Average Precision(mAP)	0.1143

To address this issue, we follow the same approach like the imbalanced case, we will create 48 new samples from each existing image by cropping different parts of the image. The new pool of images will have distribution per class as figure-25. The new dataset is much more enriching compared with the original one.

Training YOLO3 on the new augmented data and validating on the validation dataset again we have a surprising result. Performance on validation dataset improves from at just 0.1143 to 0.6773mAP and the enhancements take place in all classes, this is a substantial improvement. So data augmentation again proves its efficiency in situations when data is limited.

Table 4 - Few-shot - Testing results when training data use and do not use augmentation

Class	AP	
	Original set	Augmented set
Elephant	0.0281	0.6270
Giraffe	0.0266	0.6311
Grants Gazelle	0.2882	0.7739
Mean Average Precision(mAP)	0.1143	0.6773

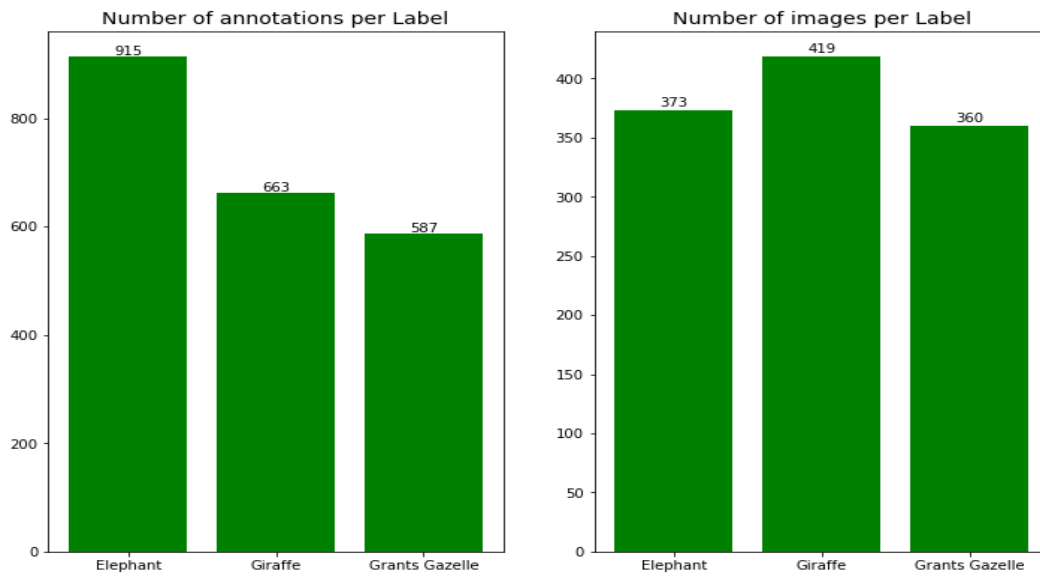


Figure 25 - Few-shot - Data distribution among class before augmentation

7 Conclusion

7.1 The effects of imbalanced classes and few-shot learning:

Training the models often requires many samples. In practice, there are situations where the quality of data may be not very good such as data not distributed well between classes (imbalanced issue) or we have very limited data (few-shot) to fit the model. In such cases, performance of the model would be negatively impacted and usually low as shown in table 1 and 3. In such scenarios, there are a possible number of solutions to address this issue: collecting more data, up-sampling or down-sampling data, making more synthetic data or even inventing different algorithms to work with such data. In our dissertation we demonstrate that image augmentation is a simple but effective method to address both issues imbalanced classes and few-shot. Table 2 and 4 show that by using image augmentation techniques, we can raise overall accuracy from just 0.06mAP to 0.38mAP for class imbalance, and from 0.11mAP to 0.67mAP for few-shot learning.

It is worth noting that the Camera Trap dataset has high resolution and multiple objects in images with diversified sizes, detecting objects in such a dataset is often harder. For datasets with lower size, lesser objects in each image, non-overlapping between objects, we can expect that our model will have better performance.

7.2 Limitation:

YOLO3 is a good model in object detection, its high prediction speed and comparable accuracy make it suitable for applications where real time prediction is required. The model is built from convolutional blocks, up-sampling blocks and custom YOLO layers. Implementing the model is a complicated task which requires a deeply understanding of high level concepts in developing models in Keras and high level of coding skills. In this project, I have presented detailed workflow, concepts as well as code for important modules such as generator, training, and evaluation.

YOLO3 has some limitations: Its prediction ability is limited to three objects in the same cell. In case there are more than three objects in a cell then the model will miss to predict additional objects. Furthermore, if two objects of the same cell were assigned to the same prior bounding box then only one is retained, the other is ignored. In practice, we hope that these scenarios are rarely met and the performance of the model will not be impacted.

7.3 Future work:

While we are working on YOLO3, newer and better models such as YOLO4, YOLO5, YOLO6 have already been released. So we should move forward to study newer and better models. Some data generation methods that we have not explored in this dissertation such as Generative adversarial network (GANs), rotating images in different angles may be also potential solution to address imbalanced and few-shot learning.

Few-shot learning is a new emerging area in computer vision. Meta-learning has progressed in recent years (Parnami and Lee, 2022). It is worth to discover more about algorithms in this approach in order to understand the underlying techniques which may help to create new

algorithms in the future. Some of them are Siamese Networks, Matching Networks, Prototypical Networks, Relation Networks, MAML, MANN, MM-Net, Meta-Networks, Cross Domain FSL.

References

- Beery, S., no date. *Caltech Camera Traps*. [online]. Available at: <<https://beerys.github.io/CaltechCameraTraps/>> [Accessed April 2022].
- Ebrahimi, M.S., Abadi, H.K., 2018. Study of Residual Networks for Image Recognition. [pdf] arxiv. Available at: <https://arxiv.org/pdf/1805.00325.pdf> [Accessed 10 May 2022]
- IBM, no date. *What is computer vision*. [online] IBM. Available at: <<https://www.ibm.com/topics/computer-vision>> [Accessed March 2022].
- Jiang, X., Hadid, A., Pang, Y., Granger, E. and Feng, X., 2019. *Deep Learning in Object Detection and Recognition*. Springer.
- Jiao, L., Zhang, F., Yang, S., Feng, Z., Qu, R., 2019. *A survey of deep learning-based object detection*. [pdf] arxiv. Available at: < <https://arxiv.org/pdf/1907.09408.pdf> > [Accessed 15 May 2022].
- Michelucci, U., 2019. *Advanced Applied Deep Learning: Convolutional Neural Networks and Object Detection*. Apress.
- Ochal, M., Patacchiola, M., Vazquez, J., Storkey, A., Wang, S., 2021. *Few-shot learning with class imbalance*. [pdf] arxiv. Available at: < <https://arxiv.org/pdf/2101.02523.pdf>> [Accessed 20 April 2022].
- O'Shea, K. and Nash, R., 2015. *An introduction to Convolutional Neural Networks*. [pdf] arxiv. Available: < <https://arxiv.org/pdf/1511.08458.pdf> > [Accessed February 2022].
- Park S., Lim, J., Jeon, Y., Choi J., 2021. *Influence-Balanced Loss for Imbalanced Visual Classification*. [pdf] arxiv. Available at: < <https://arxiv.org/pdf/2110.02444.pdf>> [Accessed 15 May 2022].
- Parnami, A. and Lee, M., 2022. *Learning from few examples: A summary of approaches to few-shot learning*. [pdf] Available at: < <https://arxiv.org/pdf/2203.04291.pdf> > [Accessed 20 July 2022].
- Redmon, J. and Fahadi, A., 2018. *YOLOv3: An Incremental Improvement*. [pdf] arxiv. Available at: < <https://arxiv.org/pdf/1804.02767.pdf> > [Accessed January 2022].
- Redmon, J., Divvala, S., Girshick, S., Farhadi, A., 2016. *You Only Look Once: Unified, Real Time Object Detection*. [pdf] arxiv. Available at: < <https://arxiv.org/pdf/1506.02640.pdf>> [Accessed 15 March 2022]
- Yang, S., Xiao, W., Zhang, M., Guo, S., Zhao, J., Shen, F., 2022. *Image Data Augmentation for deep learning*. [pdf] arxiv. Available at: < <https://arxiv.org/pdf/2204.08610.pdf>> [Accessed 1 April 2022]
- Verdhan, V., 2021. *Computer Vision Using Deep Learning: Neural Network Architectures with Python and Keras*. Apress.

Further Readings:

Keras Developer guides:

<https://keras.io/guides/>

Keras API reference:

<https://keras.io/api/>

Tensorflow API:

https://www.tensorflow.org/api_docs/python/tf

Appendix – Software Descriptions:

This appendix provides some guidelines to explore source codes of dissertation. Overall structure of code has two folders: “annotations” and “YOLO3”. Where “annotations” contain necessary work for data preparation and “YOLO3” contain source code relating YOLO3 like model architecture, training process, evaluation process, generator...For easy to understand, I list down files with descriptions in each folder in this appendix (all of these content can be found in “software description” file).

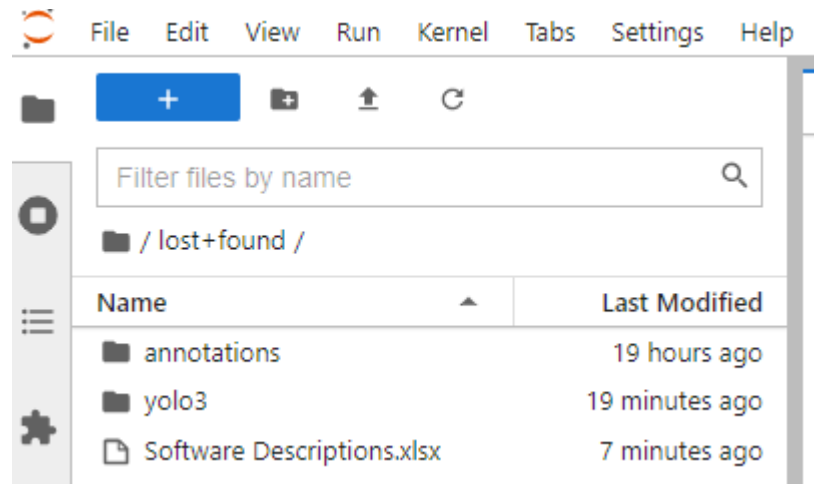


Figure 26-Overall Hierarchy

Files in annotation folders:

Files/Folder	Type	Purpose/Description
crop	Folder	Contain all cropped images from original training image
Process_original_images	Folder	Contain all cleaning work for data: - Checking available images(Available_images.ipynb) - Get coordinates of bounding boxes(Get_Coordinates_bbox.ipynb) Running these files take quite large amount of time then some csv files are generated for backup results then we do not have to run again
ann_final.csv	file	This is the final output from cleaning data process. It contain all valid information about the data: Images, annotations which will be used for our analysis process
train_valid_split.ipynb	file	Python file that read ann_final.csv and split it into train and validation set. Outputs of this work is: - imbalance_train.csv - imbalance_valid.csv These two files are copy to train and valid folder of YOLO3 respectively

imbalance_train.csv	file	A csv file contains all training data information(images and annotations) which will be used to train YOLO3 model to analyze imbalance effects
imbalance_valid.csv	file	A csv file contains all validation data information(images and annotations) which will be used to validate performance of YOLO3 for imbalance analysis(both cases: before and after augmentation training dataset)
imbalance_crop.ipynb	file	This python file generates cropped images from training data. Output of this work is a csv file imbalance_crop.csv which contain all old and new information for samples(old and new images, old and new annotations)
imbalance_crop.csv	file	A csv file contains all training data(images and annotations) after augmentation.
Few_shot.ipynb	file	This python file generates data to analyses few-shot learning. - It read imbalance_train.csv file and remove Wildebeest class then output fewshot_train.csv file(used to train YOLO3 for few-shot case) - It read imbalance_valid.csv file and remove Wildebeest class then output fewshot_valid.csv file(used to validate YOLO3 for few-shot case when no augmentation applied) - It read imbalance_crop.csv file and remove Wildebeest class then output fewshot_crop.csv file(used to train YOLO3 for few-shot case when augmentation applied)
fewshot_train.csv	file	This csv file contains all information of training data which used to train YOLO3 for few-shot test when image augmentation is not applied.
fewshot_crop.csv	file	This csv file contains all information of training data which used to train YOLO3 for few-shot test when image augmentation is applied.
fewshot_valid.csv	file	This csv file contains all information of training data which used to validate YOLO3 for few-shot test when both image augmentation is applied or not applied.
Data_analysis.ipynb	file	This file is used to generate data distribution charts among classes for different datasets: - imbalance_train.csv - imbalance_valid.csv - fewshot_train.csv - fewshot_valid.csv - fewshot_crop.csv - imbalance_crop.csv

Files in YOLO3 folders:

Files/Folder	Type	Purpose/Description
train	Folder	Contain all csv files for training YOLO3
valid	Folder	Contain all csv files for validating YOLO3
utils	Folder	Contain supporting files and functions for main modules
Imbalance_no_crop.json	file	Configuration file which provides parameters for training YOLO3 model to test imbalance effects on original data(no additional samples)
Imbalance_no_crop.h5	file	Weights of YOLO3 model when training on original imbalance data
Imbalance_crop.json	file	Configuration file which provides parameters for training YOLO3 model to test imbalance effects when we add more samples(cropped images) to training data
Imbalance_crop.h5	file	Weights of YOLO3 model when training on original imbalance data+generative data(cropped images)
Fewshot_no_crop.json	file	Configuration file which provides parameters for training YOLO3 model to test few-shot effects on original data(no additional samples)
Fewshot_no_crop.h5		Weights of YOLO3 model when training on original few-shot data
Fewshot_crop.json	file	Configuration file which provides parameters for training YOLO3 model to test few-shot effects when we add more samples(cropped images) to training data
Fewshot_crop.h5	file	Weights of YOLO3 model when training on original few-shot data+generative data(cropped images)
Imbalance_result.txt	file	Containing validating results of imbalance tests
Fewshot_result.txt	file	Containing validating results of few-shot tests
backend.h5	file	Initial weights of YOLO3 model which assist training process complete faster
generator.py	file	Generate batches and manage internal image augmentation of YOLO3 model
train.py	file	Manage training process
YOLO.py	file	contain YOLO3 model
evaluate.py	file	Used to validate model on validation set
cameratrapp_voc.py	file	The file is used to parse images and annotations
gen_anchors.py	file	The file is used to generate anchors for different training datasets
Run.ipynb	file	This file contain some commands to run train, evaluate, gen_anchors process
callbacks.py	file	support to perform actions at various stages of training like at the start or end of each epoch or batch...